

Site despre Limbaje de Programare

1. Introducere

Pentru proiectul de la Baza de Date se va realiza un site despre limbaje de programare. Ideea pornește de la o problemă observată în facultate - mulți colegi se întreabă mereu ce limbaj ar trebui să învețe primul, care sunt diferențele între ele, sau ce framework-uri există. S-a considerat că ar fi util să existe un loc unde să se găsească toate informațiile astea la un loc, fără să fie nevoie să cauți prin zeci de site-uri diferite.

Problema este că acum, dacă vrei să înveți despre un limbaj de programare, trebuie să navighezi prin tot internetul. Cauți pe Google și găsești mii de articole, tutoriale, forumuri, și fiecare spune altceva. Pentru cineva care abia începe cu programarea, poate fi destul de copleșitor. Informațiile sunt fragmentate - pe un site găsești o explicație despre ce este Python, pe altul despre aplicațiile lui practice, pe un al treilea despre framework-uri, și pe un al patrulea despre dificultatea de învățare. Acest proces de căutare și filtrare a informațiilor relevante consumă foarte mult timp și energie, timp care ar putea fi dedicat învățării efective.

Mai mult decât atât, calitatea și actualizarea informațiilor variază enorm de la o sursă la alta. Unele site-uri conțin informații învechite, altele sunt prea tehnice pentru începători, iar altele sunt prea superficiale pentru a fi cu adevărat utile. Lipsa unei structuri clare și a unui sistem de comparație între limbaje face ca procesul de decizie să fie și mai dificil. De exemplu, un începător ar vrea să știe rapid: "Care este diferența între Python și JavaScript?" sau "Ce limbaj ar trebui să învăț dacă vreau să lucrez în data science?" - întrebări simple, dar pentru care răspunsurile sunt răspândite pe zeci de pagini web diferite.

Acest site va veni să rezolve această problemă - un singur loc unde ai tot ce-ți trebuie, organizat și ușor de înțeles. Ideea este să centralizeze informațiile esențiale despre fiecare limbaj de programare într-un format consistent și accesibil, eliminând astfel nevoia de a naviga prin multiple surse și de a compara informații contradictorii.

Scopul principal este să ajute studenții și începătorii să descopere și să învețe despre diferite limbaje de programare. Practic, dacă vrei să știi totul despre Python, vei intra pe site, vei da click pe Python, și gata, vei avea tot acolo: ce este, când a apărut, pentru ce se folosește, ce framework-uri are, cât de greu este să-l înveți. Toate aceste informații vor fi prezentate într-un format unitar și ușor de parcurs, fără să fie nevoie să deschizi zeci de tab-uri în browser.

Aplicația se va adresa în special următoarelor categorii de utilizatori: studenților la informatică care abia încep și nu știu ce limbaj să aleagă pentru proiectele lor, persoanelor care vor să învețe programare de la zero și sunt copleșite de multitudinea de opțiuni disponibile, programatorilor care vor să încerce tehnologii noi și doresc o prezentare rapidă a alternativelor existente, și oricui este curios despre limbaje și vrea să compare diferite opțiuni pentru a lua o decizie informată.

Se va dori ca totul să fie simplu și prietenos. Pentru un începător poate fi intimidant când citește documentație tehnică plină de termeni complicați sau când se confruntă cu articole care presupun

cunoștințe prealabile avansate. Din acest motiv, informațiile vor fi prezentate într-un mod accesibil, fără jargon excesiv și cu explicații clare. Interfața va fi intuitivă, permițând navigarea ușoară între diferite limbaje și categorii, iar structura informațiilor va fi consistentă pentru fiecare limbaj, astfel încât utilizatorii să știe exact ce să se aștepte și unde să găsească informația dorită.

Un alt aspect important va fi posibilitatea de comparație între limbaje. Utilizatorii vor putea filtra limbajele după criterii precum nivelul de dificultate (începător, mediu, avansat) sau domeniul de utilizare (web development, mobile development, data science, game development etc.). Această funcționalitate va transforma site-ul dintr-o simplă colecție de informații într-un instrument real de luare a deciziilor, ajutând utilizatorii să găsească limbajul care se potrivește cel mai bine nevoilor și obiectivelor lor.

În plus, prin includerea informațiilor despre framework-uri și biblioteci populare pentru fiecare limbaj, aplicația va oferi și o perspectivă asupra ecosistemului tehnologic asociat. Acest lucru este esențial pentru că, în practică, alegerea unui limbaj înseamnă și alegerea unui ecosistem de tooling, comunități și resurse de învățare. Un începător trebuie să știe nu doar că Python este un limbaj bun pentru data science, ci și că are la dispoziție biblioteci precum pandas, NumPy și scikit-learn care fac analiza datelor mai simplă.

2. Descrierea Aplicației

Funcționalități principale

Site-ul va prezenta limbajele de programare sub formă de carduri organizate alfabetic pe pagina principală. Fiecare card conține logo-ul limbajului, numele complet, o scurtă descriere și informații de bază precum nivelul de dificultate și anul apariției.

Sistemul de filtrare permite utilizatorilor să găsească rapid limbajele potrivite nevoilor lor. Filtrarea după dificultate afișează doar limbajele marcate ca începător, mediu sau avansat în funcție de selecția utilizatorului. Filtrarea după domeniu caută în descrierile limbajelor și în câmpul special pentru domenii de utilizare și afișează doar rezultatele relevante pentru web development, mobile development, data science sau alte categorii selectate.

Funcția de căutare permite introducerea cuvintelor cheie care sunt căutate atât în numele limbajului cât și în descrierea sa. De exemplu căutarea după "mobile" va returna toate limbajele folosite în dezvoltarea aplicațiilor mobile chiar dacă utilizatorul nu cunoaște exact numele lor precum Swift sau Kotlin.

Pagina detaliată a fiecărui limbaj include descrierea completă care explică filozofia limbajului și contextul istoric, anul apariției pentru referință temporală și nivelul de dificultate. Partea cea mai importantă este lista completă de framework-uri și biblioteci organizate pe categorii precum frameworks pentru web development, libraries pentru data science și tools pentru testing. Framework-urile dintr-o categorie sunt sortate alfabetic.

Secțiunea de resurse educaționale conține materiale validate organizate după tip precum tutoriale video, cursuri online complete, articole și documentații oficiale. Fiecare resursă are asociat un nivel de dificultate astfel încât începătorii pot filtra doar materialele potrivite pentru ei fără a fi copleșiți de documentație tehnică avansată.

Recenziile utilizatorilor apar pe pagina fiecărui limbaj și includ un rating de la 1 la 5 stele împreună cu un comentariu text în care autorul își exprimă experiența personală. Recenziile sunt sortate de la cele mai recente la cele mai vechi pentru a reflecta starea actuală a limbajului, iar media tuturor rating-urilor este afișată vizibil în partea superioară a paginii.

Topul celor mai apreciate limbaje este generat automat pe baza mediei rating-urilor primite de la utilizatori și oferă o perspectivă asupra popularității reale.

Tipuri de utilizatori

Aplicația definește două roluri distincte cu permisiuni diferite care asigură atât accesibilitatea conținutului cât și controlul calității informațiilor.

Utilizatorii normali se pot înregistra gratuit pe platformă folosind un username unic, o adresă de email validă și o parolă care va fi hash-uită înainte de stocare în baza de date. După autentificare aceștia pot vizualiza tot conținutul disponibil, pot folosi toate funcțiile de filtrare și căutare și pot scrie recenzii pentru limbajele pe care le-au folosit. Sistemul impune restricția ca fiecare utilizator să poată scrie o singură recenzie pentru fiecare limbaj prevenind astfel spam-ul și menținând autenticitatea feedback-ului. Dacă un utilizator își schimbă părerea poate edita recenzia existentă dar nu poate adăuga una nouă pentru același limbaj.

Administratorii au toate permisiunile utilizatorilor normali plus drepturi extinse de management al conținutului. Aceștia pot adăuga limbaje noi în baza de date completând toate câmpurile necesare precum descriere, dificultate și domenii de utilizare. Pot modifica informațiile despre limbajele existente când apar actualizări sau când se descoperă inexactități în datele curente. Pot adăuga framework-uri noi pe măsură ce apar în ecosistem și pot șterge framework-uri care au devenit deprecate sau nu mai sunt folosite în industrie. De asemenea pot gestiona resursele educaționale adăugând materiale noi validate și eliminând linkuri stricate sau resurse de calitate scăzută.

Separarea permisiunilor permite platformei să crească organic prin contribuțiile comunității în forma recenziilor menținând în același timp controlul asupra calității informațiilor tehnice.

Rapoarte și statistici

Site-ul oferă câteva rapoarte care te ajută să navighezi mai ușor prin limbaje și să compari ce opțiuni ai.

1. Statistici generale ale platformei Un raport de sinteză care afișează numărul total de înregistrări pentru fiecare entitate din sistem: utilizatori, limbaje, framework-uri, resurse și mentori. Este esențial pentru administrator pentru a monitoriza dimensiunea curentă a bazei de date și ritmul de creștere al platformei.

2. Limbaje clasificate după dificultate Prezintă listele de limbaje filtrate în funcție de nivelul de accesibilitate ("începător" sau "avansat"). Acest raport ajută la verificarea echilibrului educațional al platformei, asigurând că există suficiente opțiuni atât pentru cei care abia încep programarea, cât și pentru utilizatorii experimentați.

3. Cronologia tehnologiilor Afișează extremele din baza de date: cele mai vechi limbaje (clasice) și cele mai noi tehnologii adăugate. Este util pentru a înțelege spectrul istoric acoperit de platformă și

pentru a identifica rapid tehnologiile moderne versus cele consacrate.

4. Topul mentorilor după experiență Un clasament care ordonează mentorii descrescător în funcție de numărul de ani de activitate. Ajută administratorul să identifice cei mai calificați instructori și să îi promoveze către studenții care caută ghidaj de nivel senior.

5. Cei mai recenti utilizatori Lista ultimelor conturi create pe platformă, incluzând data înregistrării și email-ul. Acest raport este folosit pentru a monitoriza noile înscrieri și pentru a valida creșterea comunității în perioadele recente.

6. Densitatea framework-urilor per limbaj O analiză complexă care numără câte framework-uri sunt asociate fiecărui limbaj de programare. Raportul evidențiază care sunt cele mai bogate ecosisteme (de exemplu, JavaScript sau Python) și care limbaje necesită adăugarea de noi biblioteci.

7. Evaluarea calității (Rating mediu) Calculează media notelor primite de fiecare limbaj din partea utilizatorilor. Spre deosebire de o simplă listă, acest raport agregat arată preferințele reale ale comunității și identifică tehnologiile cele mai apreciate de studenți.

8. Situația mentoratului activ Generează o listă cu perechile active student-mentor, filtrând doar acele sesiuni care nu au o dată de sfârșit setată. Este un instrument administrativ pentru a vedea gradul de încărcare al mentorilor și implicarea utilizatorilor în programele de învățare.

9. Resurse educaționale și materiale video Filtrează și afișează resursele disponibile, cu accent pe cele de tip video sau cele avansate. Permite administratorului să verifice rapid dacă există suficiente materiale de suport pentru utilizatorii care preferă învățarea vizuală.

10. Framework-uri specializate Un raport dedicat tehnologiilor de nișă, care extrage doar framework-urile din domenii specifice precum Data Science sau Mobile Development. Ajută la identificarea rapidă a uneltelor necesare pentru utilizatorii cu interese profesionale specifice.

3. Diagrama Entitate-Relație

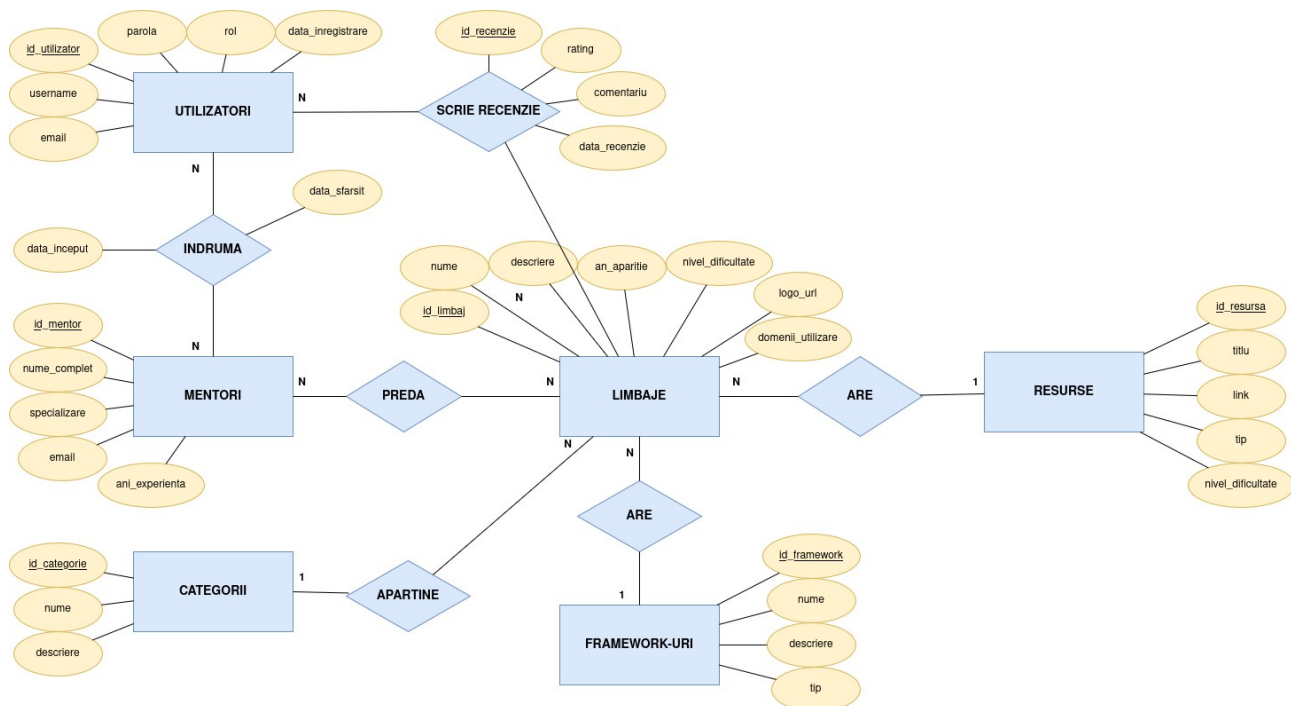


Figura 1: Diagrama Entitate-Relație a bazei de date.

Diagrama Chen ilustrează structura conceptuală a bazei de date identificând entitățile principale atributele lor și relațiile dintre ele, această reprezentare abstractă urmând să fie tradusă în tabele concrete în faza de implementare.

Entitățile principale

UTILIZATORI - Aici se găsesc toți oamenii care folosesc site-ul. Pot fi utilizatori obișnuiți care vin să citească despre limbaje și să lase recenzii, sau administratori care au grijă de conținut și adaugă informații noi. E nevoie de această separare ca nu oricine să poată șterge sau modifica ce vrea. Când cineva vrea să folosească platforma își face cont cu username, email și parolă, și după aia poate intra oricând.

LIMBAJE - Asta e partea cea mai importantă din aplicație, aici sunt toate limbajele de programare. Pentru fiecare limbaj există informații complete - când a apărut, la ce e bun, cât de greu e să-l înveți. Avem Python, JavaScript, Java, C++ și altele, fiecare cu povestea lui. Unele sunt mai bune pentru web, altele pentru mobile sau pentru lucrul cu date. Tot site-ul se învârtă în jurul limbajelor astora.

FRAMEWORK-URI - Aici se țin toate framework-urile și bibliotecile pentru fiecare limbaj. E important pentru că în realitate nimeni nu folosește doar limbajul simplu, toată lumea lucrează cu diverse framework-uri. Dacă faci web în Python folosești Django sau Flask, dacă lucrezi cu JavaScript ai React sau Vue. Un începător trebuie să știe ce opțiuni are la dispoziție, nu doar care e limbajul de bază.

RESURSE - O colecție cu materiale de învățare pentru fiecare limbaj. Sunt tutoriale video, cursuri întregi, articole utile, cărți și documentații. Materialele sunt împărțite pe nivele ca fiecare să

găsească ceva potrivit - începătorii primesc tutoriale simple, cei mai avansați găsesc lucruri mai complexe. Nu e suficient să spui cuiva să învețe Python, trebuie să-i arăți și de unde.

CATEGORII - Domeniile în care se folosesc limbajele - Web Development, Mobile Development, Data Science, Game Development și așa mai departe. Sunt utile pentru că lumea caută de obicei după ce vrea să facă nu după limbaj. Cineva zice "vreau să fac aplicații mobile" și categoriile îl ajută să vadă că ar trebui să se uite la Swift sau Kotlin. Un limbaj poate fi în mai multe categorii deodată, Python de exemplu e bun și pentru web și pentru data science.

MENTORI - Oameni cu experiență care pot să îi ajute pe alții să învețe. Fiecare mentor are specializarea lui și știi câți ani lucrează în domeniu. În loc să fie doar un site cu informații, devine mai mult o comunitate unde poți să primești sfaturi de la cineva care chiar lucrează cu tehnologiile astea și știe cum e în realitate.

Relațiile dintre entități

Relația SCRIE RECENZIE conectează UTILIZATORI și LIMBAJE într-o relație de tip multe la multe unde un utilizator poate scrie mai multe recenzii pentru limbaje diferite și un limbaj poate primi mai multe recenzii de la utilizatori diferiți. Această relație are attribute proprii care o transformă într-o entitate asociativă în implementare, attributele fiind: id_recenzie ca identificator unic, rating-ul numeric, comentariul text în care utilizatorul își exprimă părerile și data_recenzie care marchează momentul scrierii.

Relația ARE conectează LIMBAJE și FRAMEWORK-URI într-o relație de tip unu la multe unde un limbaj poate avea zero sau mai multe framework-uri asociate dar un framework aparține exact unui singur limbaj părinte. Această relație nu are attribute proprii.

Relația ARE conectează LIMBAJE și RESURSE într-o relație de tip unu la multe unde un limbaj poate avea zero sau mai multe resurse educaționale dar o resursă este dedicată unui singur limbaj specific. Această relație nu are attribute proprii.

Relația APARTINE conectează LIMBAJE și CATEGORII într-o relație de tip multe la multe unde un limbaj poate aparține la mai multe categorii deodată și o categorie poate conține mai multe limbaje diferite. De exemplu Python aparține atât categoriei Web Development cât și Data Science și Scripting. Această relație nu are attribute proprii.

Relația INDRUMA conectează MENTORI și UTILIZATORI într-o relație de tip multe la multe cu attribute proprii unde un mentor poate îndruma mai mulți utilizatori simultan și un utilizator poate avea mai mulți mentori pentru domenii diferite. Relația are attributele: data_inceput care marchează începutul colaborării și data_sfarsit care poate fi NULL dacă relația este încă activă.

Relația PREDA conectează MENTORI și LIMBAJE într-o relație de tip multe la multe unde un mentor poate preda mai multe limbaje de programare și un limbaj poate fi predat de mai mulți mentori diferiți. Această relație nu are attribute proprii și servește la identificarea expertizei fiecărui mentor în limbajele specifice pe care le cunoaște și le poate preda.

Diagrama cazurilor de utilizare

Diagrama prezintă cele patru tipuri de actori care interacționează cu sistemul și funcționalitățile la care fiecare are acces.

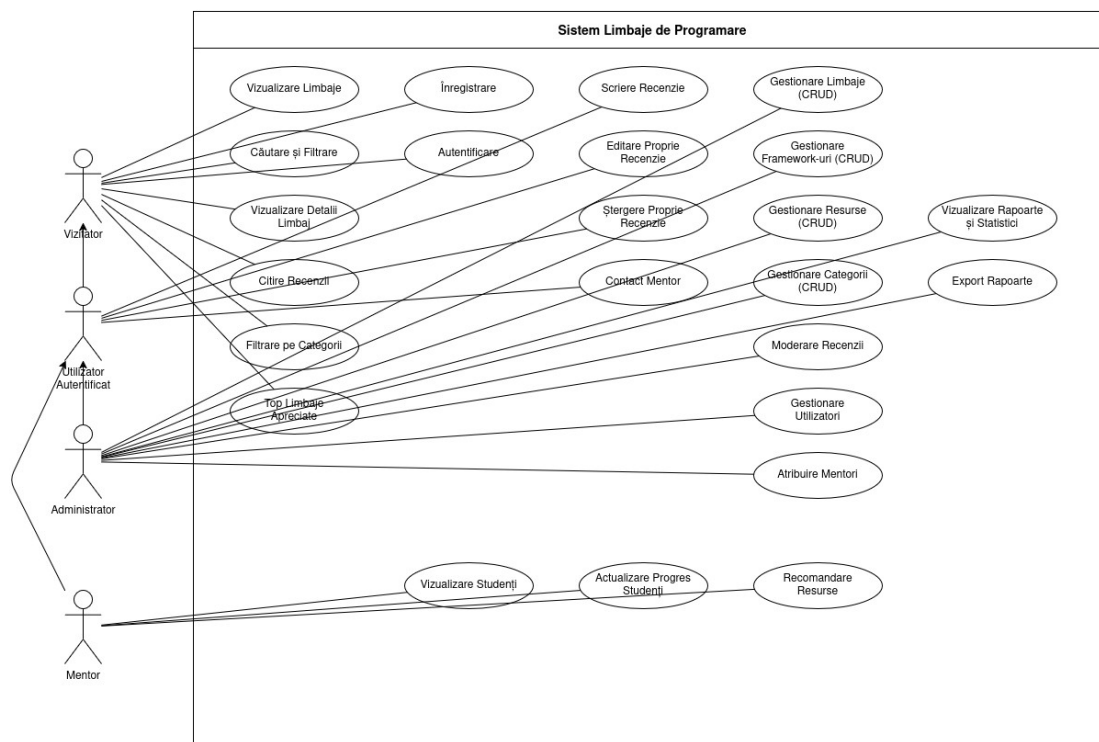


Figure 2: Diagrama de Cazuri de Utilizare (Use Case).

Vizitatorul reprezintă orice persoană care accesează site-ul fără să fie autentificată. Acesta are acces limitat la funcționalitățile platformei, putând doar să vizualizeze limbajele disponibile, să caute și să filtreze după diverse criterii, să vadă detaliile complete ale fiecărui limbaj și să citească recenziile lăsate de alți utilizatori. De asemenea poate vedea topul limbajelor apreciate și poate filtra pe categorii pentru a găsi tehnologiile care îl interesează. Practic vizitatorul poate consuma tot conținutul platformei dar nu poate contribui cu nimic până nu își face cont. Poate doar să se înregistreze pentru a deveni utilizator autentificat.

Utilizatorul Autentificat moștenește toate capacitățile vizitatorului și primește drepturi suplimentare de interacțiune. Pe lângă vizualizarea conținutului poate să scrie recenzii pentru limbajele pe care le-a folosit, oferind un rating și un comentariu cu experiența sa. Are posibilitatea să își editeze propriile recenzii dacă își schimbă părerea sau să le șteargă complet. Important este că poate scrie o singură recenzie pentru fiecare limbaj, prevenind astfel spam-ul. Mai poate contacta mentorii disponibili pe platformă pentru a primi îndrumare în învățarea unui anumit limbaj. Utilizatorul autentificat trebuie să se autentifice înainte de a accesa aceste funcționalități extinse.

Administratorul are cele mai extinse permisiuni din sistem și este responsabil cu menținerea calității conținutului. El poate gestiona limbajele prin operațiile CRUD complete - adaugă limbaje noi când apar tehnologii fresh, modifică informațiile existente când sunt actualizări și poate chiar șterge limbaje deprecate. La fel procedează și cu framework-urile și resursele educaționale, având

control total asupra acestor entități. Poate gestiona categoriile de domenii pentru a păstra organizarea clară a limbajelor. Are drepturi de moderare asupra recenziilor utilizatorilor, putând șterge cele inadecvate sau spam. Poate atribui roluri de mentor anumitor utilizatori și poate gestiona conturile tuturor utilizatorilor platformei. În plus are acces exclusiv la rapoarte și statistici detaliate despre activitatea pe site - poate vizualiza diverse metrice și poate exporta rapoarte pentru analiză. Toate funcționalitățile vizitatorului și utilizatorului autentificat îi sunt și lui disponibile.

Mentorul are un rol special pe platformă, moștenind capabilitățile utilizatorului autentificat dar adăugând funcționalități educaționale specifice. El poate vizualiza lista studenților săi, adică utilizatorii cu care are sesiuni active de mentorat. Poate actualiza progresul fiecărui student marcând ce au învățat și la ce nivel se află. Are capacitatea să recomande resurse educaționale personalizate pentru fiecare student în funcție de nevoile și nivelul acestuia. Practic mentorul acționează ca un ghid pentru utilizatorii care vor să învețe anumite limbaje de programare, oferind suport și direcție în procesul de învățare.

Relațiile dintre cei patru actori formează o ierarhie clară. Vizitatorul este actorul de bază cu drepturi minime. Utilizatorul Autentificat extinde funcționalitățile vizitatorului adăugând interactivitate prin recenzii și contact cu mentorii. Mentorul la rândul lui extinde utilizatorul autentificat cu capabilități educaționale pentru a-i ghida pe alții. Administratorul are o ramură separată de la Utilizator Autentificat cu drepturi complete de management și moderare. Această structură asigură că fiecare tip de utilizator are exact permisiunile necesare pentru rolul său fără a compromite securitatea sau calitatea platformei.

4. Implementarea Bazei de Date

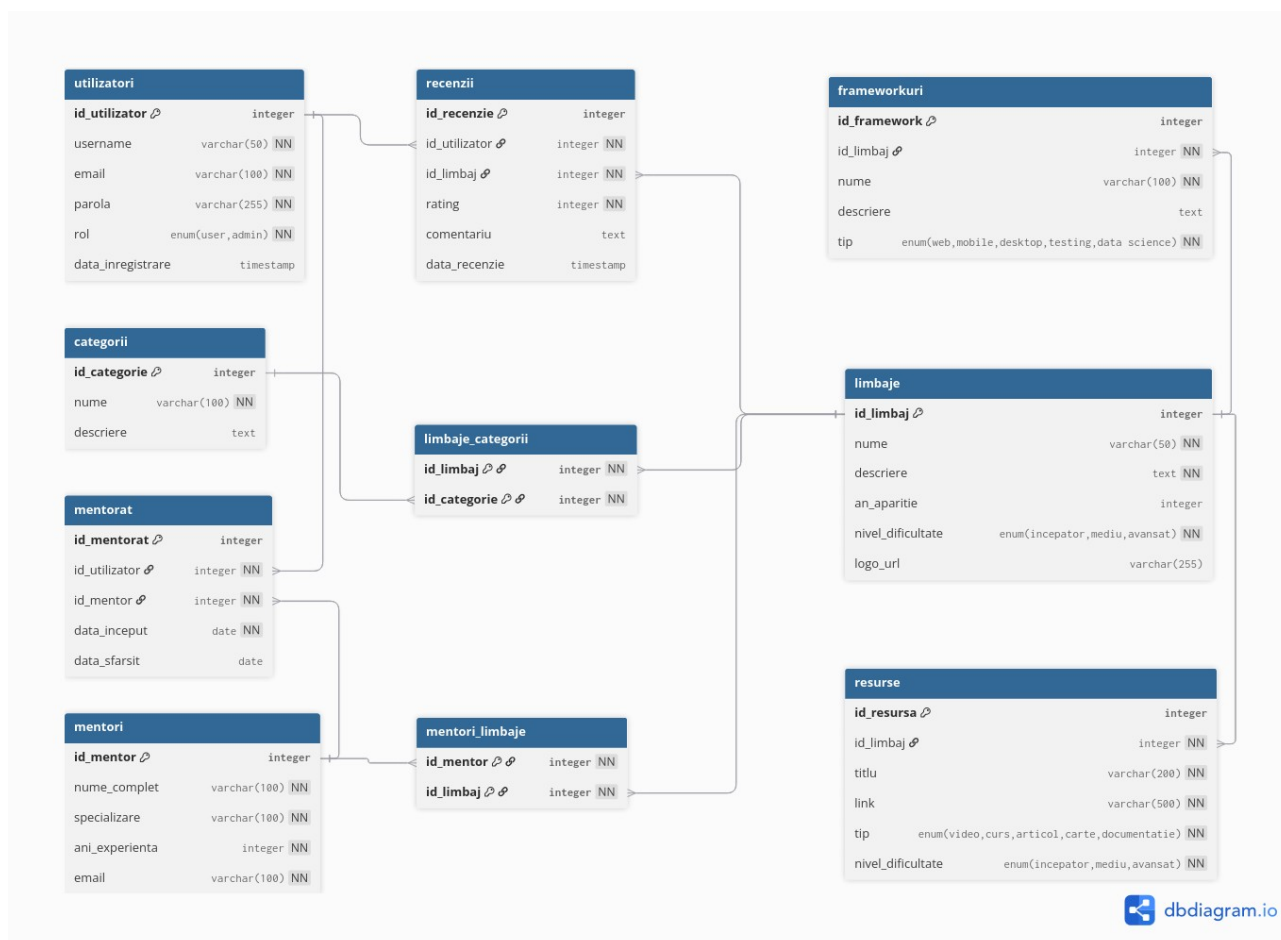


Figure 3: Schema relațională a bazei de date.

Transformarea diagramei Chen în baza de date relațională implică crearea a nouă tabele principale. Patru tabele corespund entităților de bază (UTILIZATORI LIMBAJE MENTORI și CATEGORII) celelalte cinci tabele gestionând relațiile și incluzând RECENZII pentru legătura utilizatori-limbaje FRAMEWORK-URI și RESURSE pentru relațiile unu la multe cu limbajele MENTORAT pentru legătura mentori-utilizatori și MENTORI_LIMBAJE pentru relația de predare. Mai există și tabelul de legătură LIMBAJE_CATEGORII pentru relația multe la multe.

Structura este proiectată pentru a evita redundanța datelor informațiile despre un limbaj fiind stocate o singură dată în tabelul LIMBAJE și apoi referențiate prin id în celelalte tabele, astfel actualizarea informațiilor se face într-un singur loc.

Tabelele principale

Tabelul UTILIZATORI conține câmpul `id_utilizator` ca și cheie primară de tip întreg cu auto-increment. Username-ul este `varchar(50)` cu constrângere de unicitate și NOT NULL email-ul este `varchar(100)` tot cu unicitate și NOT NULL. Parola este `varchar(255)` pentru a permite stocarea hash-urilor generate de algoritmi. Rolul este un enum cu valorile `user` și `admin` având valoarea default `user` iar `data_inregistrare` este timestamp cu valoarea default `CURRENT_TIMESTAMP`.

Tabelul LIMBAJE conține `id_limbaaj` ca și cheie primară cu auto-increment numele este `varchar(50)` cu unicitate și NOT NULL descrierea este text pentru a permite conținut mai lung fără limitare strictă. `An_aparitie` este integer pentru a stoca anul în format numeric iar `nivel_dificultate` este enum cu valorile `incepator` `mediu` și `avansat` marcate NOT NULL. `Logo_url` este `varchar(255)` pentru a stoca calea către imagine.

Tabelul FRAMEWORK-URI conține `id_framework` ca și cheie primară cu auto-increment `id_limbaaj` este integer NOT NULL și cheie străină către tabelul LIMBAJE cu ON DELETE CASCADE pentru a șterge automat framework-urile când limbaajul părinte este eliminat. Numele este `varchar(100)` NOT NULL descrierea este text iar tipul este enum cu valorile `web` `mobile` `desktop` `testing` și `data science` marcate NOT NULL. Există o constrângere de unicitate compusă pe perechea (`id_limbaaj`, `nume`) pentru a preveni duplicarea framework-urilor cu același nume pentru același limbaaj.

Tabelul RESURSE conține `id_resursa` ca și cheie primară cu auto-increment `id_limbaaj` este integer NOT NULL și cheie străină către LIMBAJE cu ON DELETE CASCADE. Titlul este `varchar(200)` NOT NULL iar link-ul este `varchar(500)` NOT NULL pentru a acomoda URL-uri lungi. Tipul este enum cu valorile `video` `curs` `articol` `carte` și `documentatie` marcate NOT NULL `nivel_dificultate` este enum cu valorile `incepator` `mediu` și `avansat` marcate NOT NULL.

Tabelul CATEGORII conține `id_categorie` ca și cheie primară cu auto-increment numele este `varchar(100)` cu constrângere de unicitate și NOT NULL iar descrierea este text pentru explicații detaliate despre domeniu.

Tabelul MENTORI conține `id_mentor` ca și cheie primară cu auto-increment `nume_complet` este `varchar(100)` NOT NULL specializarea este `varchar(100)` NOT NULL pentru a indica expertiza `ani_experienta` este integer NOT NULL cu o constrângere CHECK pentru a asigura că valoarea este pozitivă iar email-ul este `varchar(100)` cu unicitate și NOT NULL.

Tabelul RECENZII conține `id_recenzie` ca și cheie primară cu auto-increment `id_utilizator` este integer NOT NULL și cheie străină către UTILIZATORI cu ON DELETE CASCADE `id_limbaaj` este integer NOT NULL și cheie străină către LIMBAJE cu ON DELETE CASCADE. Rating-ul este integer NOT NULL cu constrângere CHECK pentru a asigura că valoarea este între 1 și 5 comentariul este text iar `data_recenzie` este timestamp cu valoarea default `CURRENT_TIMESTAMP`. Există o constrângere de unicitate compusă pe perechea (`id_utilizator`, `id_limbaaj`) pentru a impune regula unei singure recenzii per utilizator per limbaaj.

Tabelul LIMBAJE_CATEGORII este un tabel de legătură simplu care conține doar `id_limbaaj` și `id_categorie` ambele fiind NOT NULL și chei străine către tabelele respective, cele două câmpuri formând împreună cheia primară compusă asigurând unicitatea combinațiilor. Ambele chei străine au ON DELETE CASCADE pentru a elimina automat legăturile când o entitate părinte este ștearsă.

Tabelul MENTORAT conține id_mentorat ca și cheie primară cu auto-increment id_utilizator este integer NOT NULL și cheie străină către UTILIZATORI cu ON DELETE CASCADE id_mentor este integer NOT NULL și cheie străină către MENTORI cu ON DELETE CASCADE. Data_inceput este date NOT NULL iar data_sfarsit este date și poate fi NULL pentru sesiunile active. Există o constrângere CHECK pentru a asigura că data_sfarsit este NULL sau este mai mare sau egală cu data_inceput.

Tabelul MENTORI_LIMBAJE este un tabel de legătură pentru relația PREDA și conține doar id_mentor și id_limbaaj ambele fiind integer NOT NULL și chei străine către tabelele MENTORI respectiv LIMBAJE. Cele două câmpuri formează împreună cheia primară compusă pentru a asigura că un mentor nu poate fi asociat de mai multe ori cu același limbaj. Ambele chei străine au ON DELETE CASCADE astfel încât dacă un mentor sau un limbaj este șters legăturile corespunzătoare dispar automat.

Formele normale

Normalizarea bazei de date asigură că structura tabelelor respectă anumite reguli care elimină redundanța datelor și previne anomaliile la inserare, actualizare sau ștergere. Pentru proiectul curent s-a verificat conformitatea cu primele trei forme normale și forma normală Boyce-Codd.

Forma normală 1

Pentru ca o tabelă să fie în prima formă normală trebuie ca toate valorile din coloane să fie atomice, adică fiecare celulă să conțină o singură valoare indivizibilă, și să nu existe grupuri repetate de coloane.

Tabelul UTILIZATORI respectă 1NF deoarece fiecare coloană conține o singură valoare - un username, un email, o parolă hash-uită. Nu există situații în care în câmpul email să fie stocate mai multe adrese separate prin virgulă sau alte delimitatori.

Tabelul LIMBAJE a necesitat o modificare pentru a respecta 1NF. Inițial câmpul domenii_utilizare putea conține valori multiple separate prin virgulă precum "Web Development, Mobile Development, Data Science" ceea ce încalca regula atomicității. Pentru rezolvarea acestei probleme câmpul domenii_utilizare a fost eliminat complet din structura tabelului. Relația dintre limbaje și domeniile de utilizare este acum gestionată exclusiv prin tabelul de legătură LIMBAJE_CATEGORII unde fiecare asociere limbaj-categorie apare pe un rând separat. Astfel dacă Python aparține la trei categorii diferite vor exista trei înregistrări distincte în tabelul de legătură, câte una pentru fiecare domeniu, respectând astfel principiul atomicității valorilor.

Tabelul FRAMEWORK-URI respectă 1NF pentru că fiecare framework are un singur nume, o singură descriere și aparține unui singur tip din enumerare. Nu există coloane care să conțină liste de valori.

Tabelul RESURSE respectă 1NF deoarece fiecare resursă are un titlu singular, un link singular, un tip și un nivel de dificultate. Toate valorile sunt atomice și indivizibile.

Tabelul RECENZII respectă 1NF având câmpuri atomice - un rating numeric, un comentariu text și o dată. Nu există cazuri în care un utilizator să poată da mai multe rating-uri pentru aceeași recenzie.

Toate tabelele de legătură (LIMBAJE_CATEGORII, MENTORAT, MENTORI_LIMBAJE) respectă automat 1NF deoarece conțin doar chei străine și eventual attribute simple precum date, fără structuri complexe sau valori multiple.

Forma normală 2

Pentru ca o tabelă să fie în a doua formă normală trebuie să fie deja în 1NF și toate attributele non-cheie să depindă complet de cheia primară, nu doar de o parte din ea. Această regulă se aplică în principal la tabelele care au chei primare compuse.

Tabelul UTILIZATORI are cheia primară simplă id_utilizator și toate celelalte câmpuri (username, email, parola, rol, data_inregistrare) depind complet de acest identificator unic. Nu există dependențe parțiale posibile.

Tabelul LIMBAJE are cheia primară simplă id_limbaj și toate attributele sale (nume, descriere, an_aparitie, nivel_dificultate, logo_url) sunt determinate complet de acest id. Descrierea nu poate depinde doar de nume independent de id, toate attributele depind de întreaga cheie primară.

Tabelul FRAMEWORK-URI are cheie primară simplă id_framework deci nu poate avea probleme de dependență parțială. Toate attributele (id_limbaj, nume, descriere, tip) depind complet de id_framework.

Tabelul RECENZII are cheia primară simplă id_recenzie. Deși există și o constrângere de unicitate compusă pe (id_utilizator, id_limbaj), cheia efectivă folosită este id_recenzie. Rating-ul, comentariul și data depind complet de această cheie primară simplă.

Tabelul LIMBAJE_CATEGORII are cheie primară compusă (id_limbaj, id_categorie) și nu conține alte attribute non-cheie deci nu poate avea probleme de dependență parțială. Este automat în 2NF fiind un tabel de legătură pur.

Tabelul MENTORAT are cheie primară simplă id_mentorat iar data_inceput și data_sfarsit depind complet de acest identificator. Chiar dacă există și chei străine către utilizatori și mentori acestea fac parte din conținutul tabelului nu din cheia primară.

Toate tabelele respectă 2NF deoarece fie au chei primare simple unde conceptul de dependență parțială nu se aplică, fie sunt tabele de legătură fără attribute non-cheie suplimentare.

Forma normală 3

Pentru ca o tabelă să fie în a treia formă normală trebuie să fie în 2NF și să nu existe dependențe tranzitive, adică niciun atribut non-cheie să nu depindă de alt atribut non-cheie.

Tabelul UTILIZATORI respectă 3NF deoarece username, email, parola, rol și data_inregistrare depind direct de id_utilizator fără dependențe între ele. Username-ul nu determină email-ul și invers chiar dacă ambele sunt unice, fiecare are o relație directă doar cu cheia primară.

Tabelul LIMBAJE respectă 3NF pentru că toate atributele depind doar de id_limbaj. Nu există cazuri în care nivel_dificultate să determine an_aparitie sau logo_url să depindă de nume. Toate atributele sunt independente între ele și depind doar de cheia primară.

Tabelul FRAMEWORK-URI respectă 3NF deoarece nume, descriere și tip depind doar de id_framework. Tipul unui framework nu determină automat descrierea sa și invers. Fiecare atribut este independent funcțional de ceilalți atribuți non-cheie.

Tabelul RESURSE respectă 3NF având toate atributele dependente doar de id_resursa. Tipul resursei (video, curs, articol) nu determină automat nivel_dificultate sau titlul, fiecare resursă fiind evaluată individual pentru dificultate indiferent de formatul ei.

Tabelul MENTORI respectă 3NF pentru că nume_complet, specializarea, ani_experienta și email depind doar de id_mentor. Nu există dependență între numărul de ani de experiență și specializare - un mentor cu 10 ani experiență nu determină automat că specializarea lui este Web Development sau Java.

Tabelul RECENZII respectă 3NF deoarece rating-ul, comentariul și data depind doar de id_recenzie. Rating-ul nu determină comentariul și data nu influențează rating-ul, sunt atribute independente care descriu recenzia.

În general sistemul este bine proiectat fără dependențe tranzitive evidente între atribute, fiecare entitate având atribute care descriu direct acea entitate fără lanțuri de dependențe indirecte.

Forma normală Boyce-Codd

O tabelă este în BCNF dacă este în 3NF și pentru fiecare dependență funcțională $X \rightarrow Y$, X este o supercheie adică conține cheia primară sau este o cheie candidată. Este o versiune mai strictă a 3NF care elimină și ultimele anomalii posibile.

Tabelul UTILIZATORI respectă BCNF deoarece singurele dependențe funcționale sunt $id_utilizator \rightarrow$ toate celelalte câmpuri, $username \rightarrow id_utilizator$ și $email \rightarrow id_utilizator$. Toți determinanții ($id_utilizator$, $username$, $email$) sunt chei candidate deci superchei, respectând astfel cerința BCNF.

Tabelul LIMBAJE respectă BCNF având dependența principală $id_limbaj \rightarrow$ toate atributele și dependența $nume \rightarrow id_limbaj$ deoarece numele este marcat ca unic în schemă. Ambii determinanți sunt chei candidate astfel încât nu există dependențe funcționale care să pornească de la atribute non-cheie.

Tabelul FRAMEWORK-URI respectă BCNF. Există dependența compusă $(id_limbaj, nume) \rightarrow$ toate atributele iar această combinație este marcată ca unică în schemă deci este o cheie candidată. Cheia primară $id_framework$ determină și ea toate atributele. Nu există alte dependențe funcționale problematice.

Tabelul RECENZII respectă BCNF având dependențele $id_recenzie \rightarrow$ toate atributele și $(id_utilizator, id_limbaj) \rightarrow$ toate atributele. Ambii determinanți sunt chei - primară respectiv alternativă unică - deci toate dependențele funcționale pornesc de la superchei.

Tabelul MENTORI respectă BCNF cu dependențele $id_mentor \rightarrow$ toate atributele și $email \rightarrow id_mentor$. Ambii determinanți sunt chei candidate respectând astfel cerințele formei normale Boyce-Codd.

Tabelul LIMBAJE_CATEGORII respectă BCNF având cheia primară compusă (id_limbaj, id_categorie) care determină... nimic altceva pentru că nu există alte atribute. Este un tabel de legătură pur care respectă automat BCNF.

Toate tabelele din sistem respectă BCNF deoarece nu există situații în care un atribut non-cheie sau o combinație de atribute non-cheie să determine alte atribute. Toate dependențele funcționale pornesc de la chei primare sau chei candidate, asigurând astfel că baza de date este complet normalizată și lipsită de anomalii la operațiile de inserare, actualizare sau ștergere.

Alegerea limbajului MySQL

Pentru implementarea acestui proiect s-a ales MySQL din câteva motive simple care îl fac potrivit pentru cerințele aplicației.

MySQL e folosit de mulți oameni și există multe resurse online care ajută când apare o problemă sau când nu știi exact cum să scrii o interogare. Dacă te blochezi undeva găsești repede pe cineva care a mai trecut prin aceeași situație. Se integrează ușor cu limbaje ca Python sau PHP care sunt uzuale în dezvoltarea web, deci nu e complicat să-l conectezi la restul aplicației.

Un alt motiv important e că MySQL suportă chei străine prin motorul InnoDB. Asta contează pentru că în baza asta sunt multe legături între tabele, utilizatori care scriu recenzii, limbaje care au framework-uri, resurse asociate cu limbaje. Fără suport pentru chei străine ar fi mai complicat să menții datele consistente și să eviți erorile când ștergi sau modifizi înregistrări.

MySQL vine și cu Workbench unde poți vizualiza tabelele, modifica structura bazei de date și rula interogări fără să scrii tot în linia de comandă. Pentru cineva care nu lucrează zilnic cu baze de date asta e destul de util, mai ales când vrei să verifici repede ceva sau să vezi cum arată datele.

Crearea bazei de date in MySQL

```
CREATE DATABASE IF NOT EXISTS limbaje_programare;

USE limbaje_programare;

CREATE TABLE UTILIZATORI (
    id_utilizator INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    parola VARCHAR(255) NOT NULL,
    rol ENUM('user', 'admin') NOT NULL DEFAULT 'user',
    data_inregistrare TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE CATEGORII (  
    id_categorie INT PRIMARY KEY AUTO_INCREMENT,  
    nume VARCHAR(100) NOT NULL UNIQUE,  
    descriere TEXT  
);
```

```
CREATE TABLE LIMBAJE (  
    id_limbaj INT PRIMARY KEY AUTO_INCREMENT,  
    nume VARCHAR(50) NOT NULL UNIQUE,  
    descriere TEXT,  
    an_aparitiei INT,  
    nivel_dificultate ENUM('incepator', 'mediu', 'avansat') NOT NULL,  
    logo_url VARCHAR(255)  
);
```

```
CREATE TABLE FRAMEWORK_URI (  
    id_framework INT PRIMARY KEY AUTO_INCREMENT,  
    id_limbaj INT NOT NULL,  
    nume VARCHAR(100) NOT NULL,  
    descriere TEXT,  
    tip ENUM('web', 'mobile', 'desktop', 'testing', 'data science') NOT NULL,  
    FOREIGN KEY (id_limbaj) REFERENCES LIMBAJE(id_limbaj) ON DELETE  
CASCADE,  
    UNIQUE (id_limbaj, nume)  
);
```

```
CREATE TABLE RESURSE (  
    id_resursa INT PRIMARY KEY AUTO_INCREMENT,  
    id_limbaj INT NOT NULL,  
    titlu VARCHAR(200) NOT NULL,  
    link VARCHAR(500) NOT NULL,  
    tip ENUM('video', 'curs', 'articol', 'carte', 'documentatie') NOT NULL,  
    nivel_dificultate ENUM('incepator', 'mediu', 'avansat') NOT NULL,  
    FOREIGN KEY (id_limbaj) REFERENCES LIMBAJE(id_limbaj) ON DELETE  
CASCADE  
);
```

```
CREATE TABLE RECENZII (  
    id_recenzii INT PRIMARY KEY AUTO_INCREMENT,  
    id_utilizator INT NOT NULL,  
    id_limbaj INT NOT NULL,  
    rating INT NOT NULL,  
    comentariu TEXT,  
    data_recenzii TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (id_utilizator) REFERENCES UTILIZATORI(id_utilizator) ON  
DELETE CASCADE,  
    FOREIGN KEY (id_limbaj) REFERENCES LIMBAJE(id_limbaj) ON DELETE  
CASCADE,  
    UNIQUE (id_utilizator, id_limbaj),  
    CHECK (rating >= 1 AND rating <= 5)  
);
```

```
CREATE TABLE LIMBAJE_CATEGORII (  
    id_limbaj INT NOT NULL,  
    id_categorie INT NOT NULL,  
    PRIMARY KEY (id_limbaj, id_categorie),  
    FOREIGN KEY (id_limbaj) REFERENCES LIMBAJE(id_limbaj) ON DELETE  
CASCADE,  
    FOREIGN KEY (id_categorie) REFERENCES CATEGORII(id_categorie) ON  
DELETE CASCADE  
);
```

```
CREATE TABLE MENTORI (  
    id_mentor INT PRIMARY KEY AUTO_INCREMENT,  
    nume_complet VARCHAR(100) NOT NULL,  
    specializarea VARCHAR(100) NOT NULL,  
    ani_experienta INT NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    CHECK (ani_experienta > 0)  
);
```

```
CREATE TABLE MENTORAT (  
    id_mentorat INT PRIMARY KEY AUTO_INCREMENT,  
    id_utilizator INT NOT NULL,  
    id_mentor INT NOT NULL,  
    data_inceput DATE NOT NULL,  
    data_sfarsit DATE,  
    FOREIGN KEY (id_utilizator) REFERENCES UTILIZATORI(id_utilizator) ON  
DELETE CASCADE,  
    FOREIGN KEY (id_mentor) REFERENCES MENTORI(id_mentor) ON DELETE  
CASCADE,  
    CHECK (data_sfarsit IS NULL OR data_sfarsit >= data_inceput)  
);
```



```
CREATE TABLE MENTORI_LIMBAJE (  
    id_mentor INT NOT NULL,  
    id_limbaaj INT NOT NULL,  
    PRIMARY KEY (id_mentor, id_limbaaj),  
    FOREIGN KEY (id_mentor) REFERENCES MENTORI(id_mentor) ON DELETE  
CASCADE,  
    FOREIGN KEY (id_limbaaj) REFERENCES LIMBAJE(id_limbaaj) ON DELETE  
CASCADE  
);
```

Construirea rapoartelor in MySQL

Pentru generarea raportului complex, aplicația execută 30 de interogări SQL distincte, acoperind statistici, filtrări, sortări și analize relaționale. Mai jos sunt prezentate interogările utilizate și explicația lor:

1. Total utilizatori înregistrați

```
SELECT COUNT(*) FROM UTILIZATORI;
```

Numără toate înregistrările din tabelul de utilizatori pentru a monitoriza dimensiunea comunității.

2. Număr total de limbaje

```
SELECT COUNT(*) FROM LIMBAJE;
```

Returnează numărul de limbaje de programare documentate în baza de date.

3. Total framework-uri disponibile

```
SELECT COUNT(*) FROM FRAMEWORK_URI;
```

O statistică generală despre volumul de biblioteci și framework-uri stocate.

4. Resurse educaționale

```
SELECT COUNT(*) FROM RESURSE;
```

Indică numărul total de materiale de învățare (cursuri, tutoriale) disponibile.

5. Mentori activi

```
SELECT COUNT(*) FROM MENTORI;
```

Numără câți instructori sunt disponibili în platformă pentru a ghida studenții.

6. Recenzii lăsate de utilizatori

```
SELECT COUNT(*) FROM RECENZII;
```

Măsoară gradul de interacțiune al utilizatorilor prin numărul total de feedback-uri oferite.

7. Categori (Domenii)

```
SELECT COUNT(*) FROM CATEGORII;
```

Arată câte domenii de activitate (ex: Web, Mobile) sunt acoperite de platformă.

8. Număr administratori

```
SELECT COUNT(*) FROM UTILIZATORI WHERE rol = 'admin';
```

Filtrează tabelul de utilizatori pentru a vedea câți au privilegii de administrare.

9. Limbaje pentru începători

```
SELECT nume, an_aparitie  
FROM LIMBAJE WHERE nivel_dificultate = 'incepator';
```

Selectează doar limbajele marcate ca fiind accesibile celor fără experiență anterioară.

10. Limbaje nivel avansat

```
SELECT nume, an_aparitie  
FROM LIMBAJE WHERE nivel_dificultate = 'avansat';
```

Identifică limbajele complexe care necesită cunoștințe profunde de programare.

11. Limbaje clasice (înainte de 2000)

```
SELECT nume, an_aparitie  
FROM LIMBAJE WHERE an_aparitie < 2000;
```

Filtrează limbajele create înainte de anul 2000 pentru a evidenția tehnologiile consacrate.

12. Framework-uri de tip Web

```
SELECT nume, descriere  
FROM FRAMEWORK_URI WHERE tip = 'web';
```

Extrage lista framework-urilor dedicate dezvoltării web (frontend sau backend).

13. Resurse Video Disponibile

```
SELECT titlu, link  
FROM RESURSE WHERE tip = 'video';
```

Filtrează resursele pentru a afișa doar tutorialele video, preferate de mulți studenți.

14. Mentori Seniori (>5 ani experiență)

```
SELECT nume_complet, ani_experienta  
FROM MENTORI WHERE ani_experienta > 5;
```

Identifică instructorii cu o experiență vastă în domeniu.

15. Recenzii de 5 stele

```
SELECT comentariu, rating  
FROM RECENZII WHERE rating = 5;
```

Afișează feedback-ul pozitiv maxim pentru a promova conținutul de calitate.

16. Cel mai vechi limbaj

```
SELECT nume, an_aparitie  
FROM LIMBAJE ORDER BY an_aparitie  
ASC LIMIT 1;
```

Ordonează cronologic limbajele și îl returnează pe primul apărut.

17. Cel mai nou limbaj

```
SELECT nume, an_aparitie  
FROM LIMBAJE ORDER BY an_aparitie  
DESC LIMIT 1;
```

Identifică cea mai recentă tehnologie adăugată în baza de date.

18. Top 3 Mentori după experiență

```
SELECT nume_complet, ani_experienta  
FROM MENTORI ORDER BY ani_experienta  
DESC LIMIT 3;
```

Creează un clasament al celor mai experimentați 3 mentori.

19. Ultimii 5 utilizatori înscriși

```
SELECT username, email, data_inregistrare  
FROM UTILIZATORI ORDER BY data_inregistrare DESC LIMIT 5;
```

Permite administratorului să vadă cine sunt cei mai noi membri ai comunității.

20. 5 Resurse Avansate

```
SELECT titlu  
FROM RESURSE WHERE nivel_dificultate = 'avansat'  
LIMIT 5;
```

Selectează un eșantion de materiale complexe pentru studenții avansați.

21. Număr framework-uri per limbaj

```
SELECT l.num, COUNT(f.id_framework)  
FROM LIMBAJE l LEFT JOIN FRAMEWORK_URI f ON l.id_limbaj = f.id_limbaj  
GROUP BY l.num;
```

Folosește un LEFT JOIN pentru a număra câte framework-uri are asociate fiecare limbaj.

22. Lista Framework-uri și Limbajul Părinte

```
SELECT f.num, l.num, f.tip  
FROM FRAMEWORK_URI f  
JOIN LIMBAJE l ON f.id_limbaj = l.id_limbaj;
```

Afișează numele framework-ului alături de limbajul din care provine.

23. Număr Resurse per Limbaj

```
SELECT l.num, COUNT(r.id_resursa)  
FROM LIMBAJE l  
LEFT JOIN RESURSE r ON l.id_limbaj = r.id_limbaj  
GROUP BY l.num;
```

Analizează care limbaje sunt cele mai bine documentate prin numărarea resurselor.

24. Rating Mediu per Limbaj

```
SELECT l.num, AVG(r.rating)  
FROM LIMBAJE l  
JOIN RECENZII r ON l.id_limbaj = r.id_limbaj  
GROUP BY l.num;
```

Calculează media aritmetică a notelor primite de fiecare limbaj din partea utilizatorilor.

25. Mentori și ce limbaje predau

```
SELECT m.nume_complet, l.nume  
FROM MENTORI m  
JOIN MENTORI_LIMBAJE ml ON m.id_mentor = ml.id_mentor  
JOIN LIMBAJE l ON ml.id_limbaj = l.id_limbaj;
```

Folosește tabelul de legătură pentru a arăta specializarea fiecărui mentor.

26. Utilizatori care au lăsat recenzii

```
SELECT DISTINCT u.username  
FROM UTILIZATORI u  
JOIN RECENZII r ON u.id_utilizator = r.id_utilizator;
```

Identifică utilizatorii activi care au contribuit cu cel puțin un feedback.

27. Categori și numărul de limbaje conținute

```
SELECT c.nume, COUNT(lc.id_limbaj)  
FROM CATEGORII c  
JOIN LIMBAJE_CATEGORII lc ON c.id_categorie = lc.id_categorie  
GROUP BY c.nume;
```

Arată distribuția limbajelor pe domenii (ex: câte sunt pe Web, câte pe Mobile).

28. Limbaje care au cel puțin un framework

```
SELECT DISTINCT l.nume  
FROM LIMBAJE l  
JOIN FRAMEWORK_URI f ON l.id_limbaj = f.id_limbaj;
```

Filtrează limbajele care au un ecosistem dezvoltat (au cel puțin un framework).

29. Studenți înscriși la Mentori

```
SELECT u.username, m.nume_complet  
FROM MENTORAT mt  
JOIN UTILIZATORI u ON mt.id_utilizator = u.id_utilizator  
JOIN MENTORI m ON mt.id_mentor = m.id_mentor  
WHERE mt.data_sfarsit IS NULL;
```

Afișează perechile student-mentor active în acest moment.

30. Framework-uri Specializate

```
SELECT nume  
FROM FRAMEWORK_URI WHERE tip IN ('data science', 'mobile');
```

Selectează framework-urile care aparțin unor nișe specifice de dezvoltare.